Page 2 - Answers (Q1 to Q10)

1. Why is String immutable in Java?

Ans: Once a String object is created, its value cannot be changed. Any operation (concat, replace, etc.) creates a new String object. This helps in security, thread-safety and String Pool optimization.

```
String s = "Java";
s.concat("Developer");   // does not change original string
System.out.println(s);   // Java
```

2. Why does == behave differently from equals()?

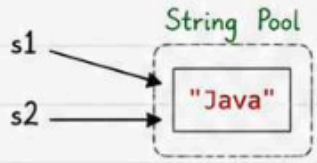
Ans:
- == compares memory reference (address of objects).
- equals() compares actual content (characters) of objects.

```
String a = new String("Java");
String b = new String("Java");
System.out.println(a == b);        // false
System.out.println(a.equals(b));   // true
```

3. What is the String Pool and why does Java use it?

Ans: String Pool is a special area in heap memory where String literals are stored. If the same literal is created again, JVM reuses the existing object from the pool instead of creating a new one. This saves memory and improves performance.

```
String s1 = "Java";
String s2 = "Java";
System.out.println(s1 == s2); // true
```



4. Why can HashMap have only one null key?

Ans: HashMap allows only one null key because internally null key is treated as a special case and mapped to bucket index 0. If multiple null keys were allowed, the previous value would be overwritten.

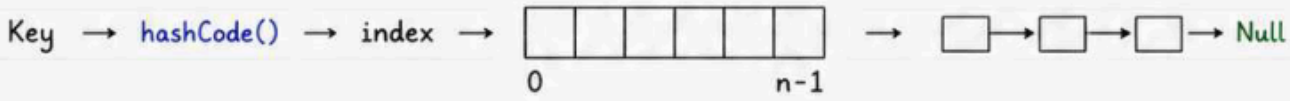
```
HashMap<String, Integer> map = new HashMap<>();
map.put(null, 1);
map.put(null, 2); // overwrites the previous value
System.out.println(map.get(null)); // 2
```

5. Why doesn't ConcurrentHashMap allow null keys or values?

Ans: ConcurrentHashMap is designed for high concurrency.
Allowing null makes it ambiguous (null could mean absent or present). To avoid this ambiguity and for better performance and safety, it does not allow null keys or values.

6. How does HashMap work internally?

Ans: HashMap uses hashing to store key-value pairs.
Key's hashCode() is calculated, then index = hash % capacity.
Collisions are handled using linked list (or tree in Java 8+).



7. Why should equals() and hashCode() be overridden together?

Ans: Hash-based collections (HashMap, HashSet) use hashCode() to find the bucket and equals() to find the exact object.
If equals() is overridden but hashCode() is not, objects with same content may be stored in different buckets.
This breaks the contract and causes unexpected behavior.

Example:

```
class User {
    String name;
    User(String name) { this.name = name; }
    @Override
    public boolean equals(Object o) {
        return ((User)o).name.equals(this.name);
    }
    // hashCode() not overridden
}
```

Two User objects with same name will be treated as different in HashSet/HashMap.

8. Why is Object the parent of every class in Java?

Ans: Object is the root class in Java.
It provides common methods like toString(), equals(), hashCode(), getClass(), wait(), notify(), etc.
Every class implicitly extends Object.

9. Why is toString() important?

Ans: toString() returns a string representation of an object.
It is useful for debugging and logging.
By default, it prints class name + hash code.

```
User u = new User("Aman");
System.out.println(u);   // calls toString()
```

10. Why is ArrayList faster than LinkedList for random access?

Ans: ArrayList is backed by an array.
Elements are stored in contiguous memory locations,
so accessing an element by index is O(1).

11. Why is LinkedList faster for insertion and deletion?

Ans: LinkedList stores elements in nodes.
When we insert or delete, we only change references
(pointers), no shifting of elements is required.
So operations are O(1).

12. Why is HashSet usually faster than TreeSet?

Ans: HashSet is backed by HashMap and provides O(1) operations
(add, remove, contains).
TreeSet is backed by a Red-Black tree and operations are O(log n)
due to sorting and balancing.

13. Why is volatile not thread-safe?

Ans: volatile guarantees visibility of changes but not atomicity.
It prevents caching issues but does not lock the variable.
So multiple threads can still modify the value together and cause race conditions.

14. Why does wait() release the lock but sleep() doesn't?

Ans:

wait()	sleep()
• Called on an object.	• Called on Thread class.
• Releases the lock of that object.	• Does NOT release any lock.
• Moves thread to waiting state.	• Just pauses the thread for given time.
• Requires notify()/notifyAll() to wake up.	• No need of notify() to resume.

15. Why should we avoid synchronizing on String literals?

Ans: String literals are stored in String Pool and are shared by all classes.
If we synchronize on them, unrelated code can block each other,
leading to performance issues or deadlocks.

Bad Example:

```
synchronized ("LOCK") {
    // critical section
}
```

Always use a private final object for synchronization.

16. Why does Java still have memory leaks despite Garbage Collection?

Ans: Garbage Collection only collects unreachable objects.
If references to unused objects are still held (directly or indirectly), GC can't remove them.
This causes memory leaks.

Example:

```
List<Object> list = new ArrayList<>();
while(true) {
    list.add(new Object());   // references never removed
}  // OutOfMemoryError eventually
```

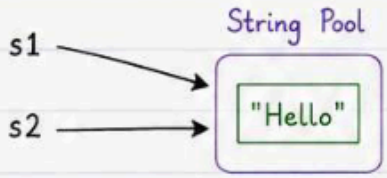
17. Why are wrapper classes immutable?

Ans: Wrapper classes (Integer, Double, etc.) are immutable so that they can be safely cached, shared and used in collections.
If they were mutable, it could break the behavior of HashMap, HashSet, etc.

18. Why are String objects stored in the String Pool?

Ans: String Pool avoids creating duplicate String objects.
It reuses existing strings, saves memory and improves performance.
Literals are stored in the pool by default.

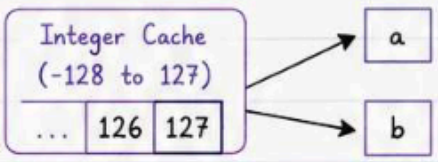
```
String s1 = "Hello";
String s2 = "Hello";
System.out.println(s1 == s2); // true
```



19. Why does Integer.valueOf(127) == Integer.valueOf(127) return true?

Ans: Integer caching is used for values between -128 to 127.
Both objects refer to the same cached object.

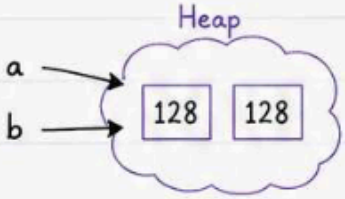
```
Integer a = Integer.valueOf(127);
Integer b = Integer.valueOf(127);
System.out.println(a == b);    // true
```



20. Why does Integer.valueOf(128) == Integer.valueOf(128) return false?

Ans: 128 is outside the cache range.
New objects are created in heap each time.

```
Integer a = Integer.valueOf(128);
Integer b = Integer.valueOf(128);
System.out.println(a == b);    // false
```



21. Why are Enums considered thread-safe?

Ans:
- Enums are immutable.
- Their instances are created only once during class loading.
- JVM guarantees safe publication.
- No state can be changed after creation.

```
enum Day { MONDAY, TUESDAY, WEDNESDAY }
```

22. Why does finally sometimes not execute?

Ans: finally may not execute if JVM shuts down before it runs.

Cases:
- System.exit(0) is called.
- JVM crashes (e.g., OutOfMemoryError, StackOverflowError).
- Kill signal (kill -9 on Linux).
- Power failure or hardware failure.

23. Why is catching Exception considered a bad practice?

Ans: Catching Exception catches everything, including programming errors like NullPointerException, ArrayIndexOutOfBoundsException, etc.

It makes debugging difficult and hides the real issue.

Always catch specific exceptions or use multi-catch.

```
try {
    // risky code
} catch (IOException | SQLException e) {
    // handle specific exceptions
}
```

24. Why should we throw specific exceptions instead of generic ones?

Ans: Specific exceptions provide clear information about the error.

It helps the caller to handle the exception properly.

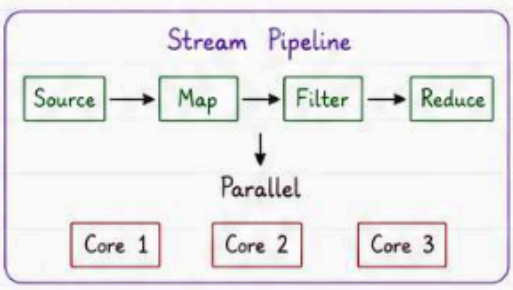
Improves code readability and maintainability.

```
throw new InvalidAgeException("Age must be greater than 18");
```

25. Why are Streams usually faster than loops for large data?

Ans: Streams use internal optimizations like:
- Lazy evaluation
- Method chaining
- Built-in parallel processing

Parallel streams can utilize multiple CPU cores.




```
List<Integer> list = ...;
int sum = list.parallelStream()
        .mapToInt(Integer::intValue)
        .sum();
```

26. Why does "+" operator behave differently for String and numbers?

Ans:
- For numbers: "+" performs arithmetic addition.
- For String: "+" performs concatenation.

```
int a = 10, b = 20;
System.out.println(a + b);       // 30
System.out.println("10" + 20);  // 1020 (String concatenation)
```

27. Why is StringBuilder preferred over StringBuffer now?

Ans: StringBuilder is non-synchronized and hence faster.
StringBuffer is synchronized, so it is slower.
Use StringBuilder unless thread-safety is required.

28. Why does HashMap allow one null key and multiple null values?

Ans: HashMap is designed to handle null gracefully.
It treats null key's hashCode as 0 and stores it in bucket 0.
Multiple null values are allowed because values are not used in hashing or key comparison.

29. Why is try-with-resources better for closing resources?

Ans: It automatically closes resources even if exceptions occur.
It reduces boilerplate code and prevents resource leaks.
Introduced in Java 7.

```
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
   // use resource
} catch (IOException e) {
   e.printStackTrace();
} // br is auto-closed
```

→ Cleaner, Safer, Better!

30. Why does equals() break the contract with hashCode()?

Ans: If two objects are equal (equals() returns true) but have different hashCode(), they may go to different buckets in HashMap/HashSet.
This breaks the contract and causes unexpected behavior.

✗ equals() true
✗ hashCode() different
=
✗ Contract broken!

